

Discovering New Theorems via LLMs with In-Context Proof Learning in Lean

OMRON SINIC X Corporation (Research Engineer) / RIKEN AIP (Visiting Scientist) / AutoRes

Kazumi Kasaura

Joint Work with Noato Onda, Yuta Oriike, Masaya Taniguchi, Akiyoshi Sannai, Sho Sonoda in AutoRes

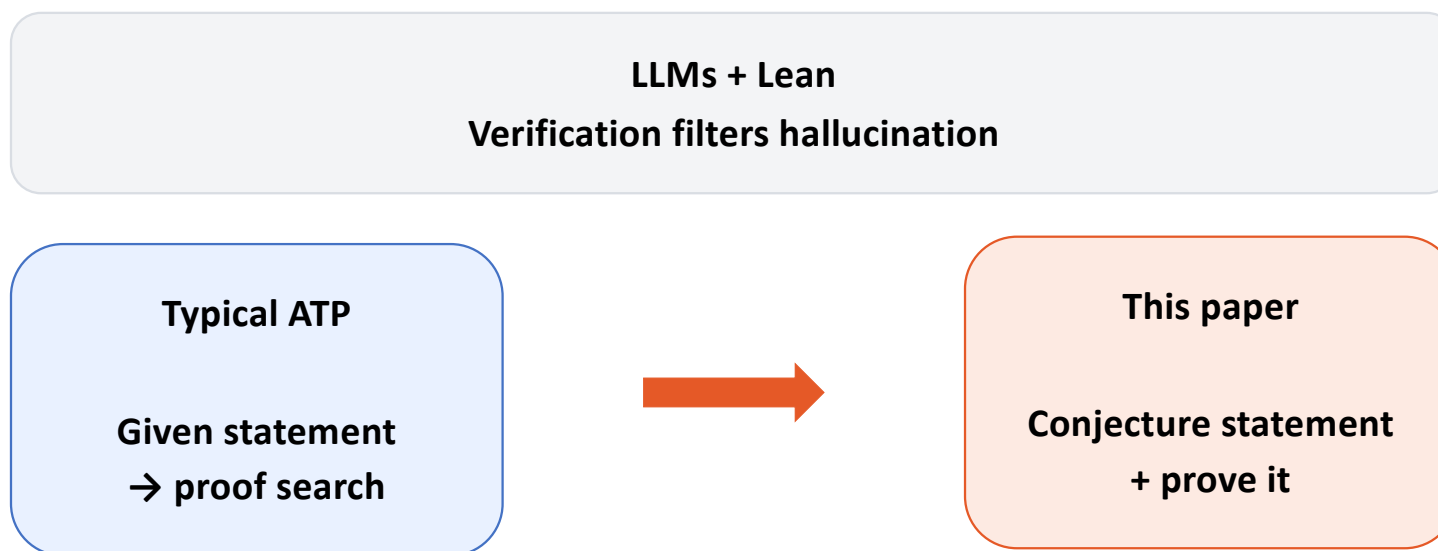
SINIC X

AUTORES

Automated Research Project

From proving problems to discovering theorems

Core question: can an LLM choose a useful theorem statement, not just prove a given one?



What the paper contributes

A pipeline, a theory of search allocation, and empirical evidence in Lean.

1

Conjecturing-Proving Loop

Generate Lean conjectures, prove them separately, and grow a verified library.

2

Search theory

Repeated proof attempts reduce the bias toward only easy-to-prove statements.

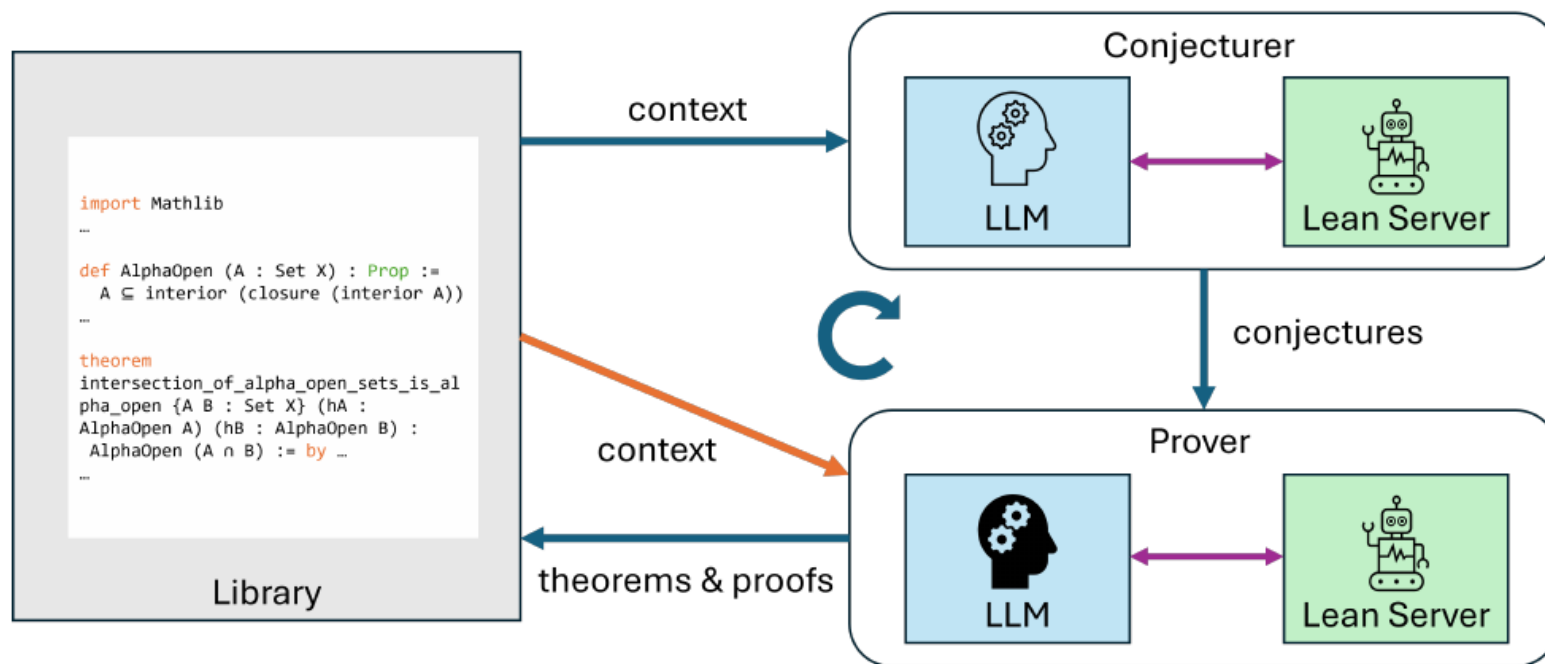
3

Experiment

CPL rediscovers an alpha-open intersection theorem and improves proof success via context.

CPL at a glance

The library is both memory and curriculum: it supplies context to the conjecturer and the prover.



Verified theorem + proof pairs are appended to the library and reused in the next iteration.

How conjecturing and proving are separated

Conjectures are filtered for validity and novelty; proofs are repaired using Lean feedback.

Conjecture loop

- I. LLM writes Lean theorem statements
- II. Lean checks syntax and novelty
- III. exact? rejects already provable statements
- IV. Valid novel conjectures go to prover

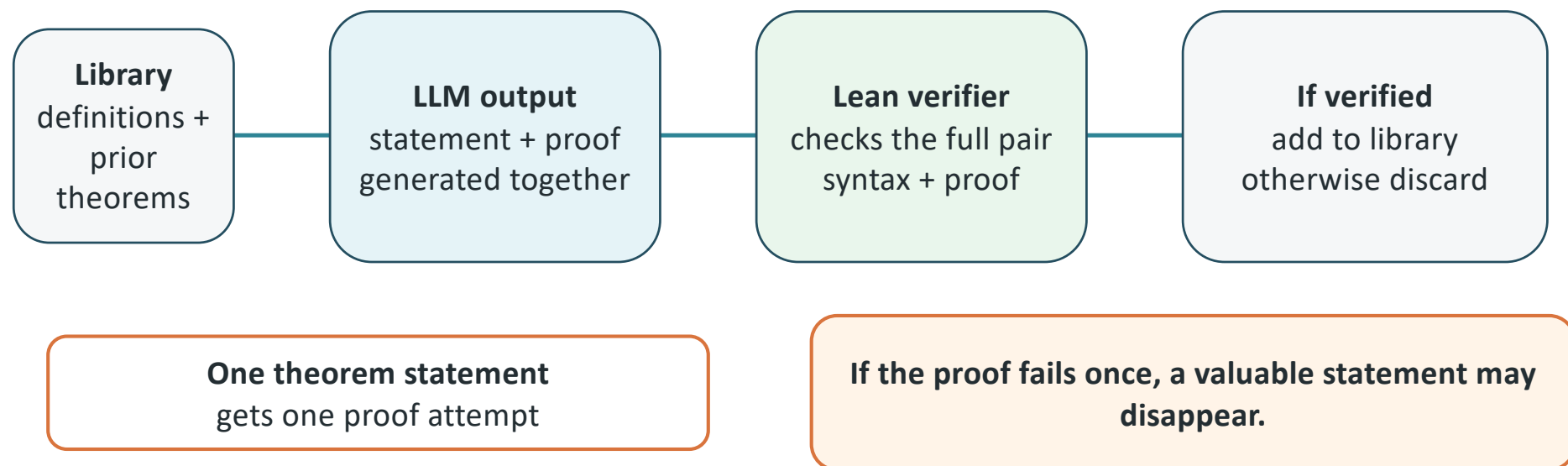
Prover loop

1. LLM writes proof code after `:=`
2. Lean verifies proof
3. Errors return to the LLM
4. Stop at success or 16 trials

Separation lets the system spend multiple proof attempts after a promising statement appears.

Baseline: Simple Loop

A single LLM must propose a statement and prove it in the same shot.



Search bias: Simple Loop rewards statements that are likely to be proposed and easy to prove immediately.

Generation probability: the one-chance penalty

Separating conjecturing from proving changes the chance that a proposed theorem survives.

$s(T)$
statement sampling

$r(T)$
one-proof success

N
proof attempts

Experiment: $N = 16$

Simple Loop
 $P_{SL}(T) \propto s(T) \cdot r(T)$

CPL
 $P_{CPL}(T) \propto s(T) \cdot [1 - (1 - r(T))^N]$

Example: hard theorem with $r(T) = 0.1$

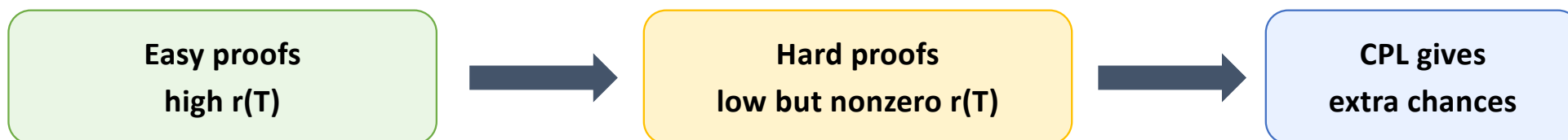
 SL: 10%

 CPL: 81%

Takeaway
More chances

Why CPL can surface harder theorems

Repeated attempts make CPL approach the distribution of provable statements as N grows.



For SL, theorem discovery is weighted by proof success: easy proofs dominate.

For CPL, proof success becomes $1-(1-r)^N$, so low- r theorems are less suppressed.

Prediction: For propositions with a sufficiently small probability of successful proof, CPL is more likely to generate it, even if it generates fewer theorems overall.

CPL should create a longer tail of verified Lean proofs.

Experimental setting: topology notions

The notions are anonymized as P1, P2, and P3 to reduce memorization effects.

P1(A)

$A \subseteq \text{closure}(\text{interior } A)$
semi-open

P2(A)

$A \subseteq \text{interior}(\text{closure}(\text{interior } A))$
alpha-open

P3(A)

$A \subseteq \text{interior}(\text{closure } A)$
preopen

Target theorem: $P2(A)$ and $P2(B) \Rightarrow P2(A \cap B)$

Why it matters

Key step toward showing alpha-open sets form another topology.

Initial library

Only definitions + Mathlib operators such as closure and interior.

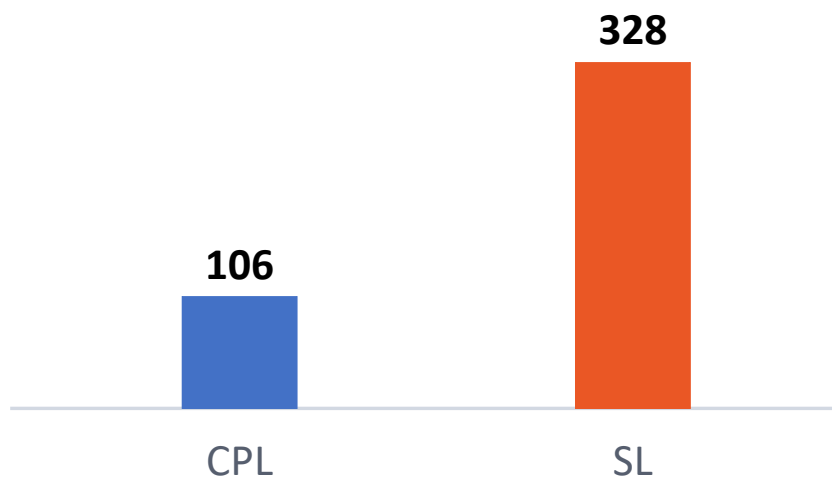
We use GPT-o3 for both Conjecturer and Prover

Main result: target theorem discovery

CPL finds the target theorem despite producing fewer total theorems.

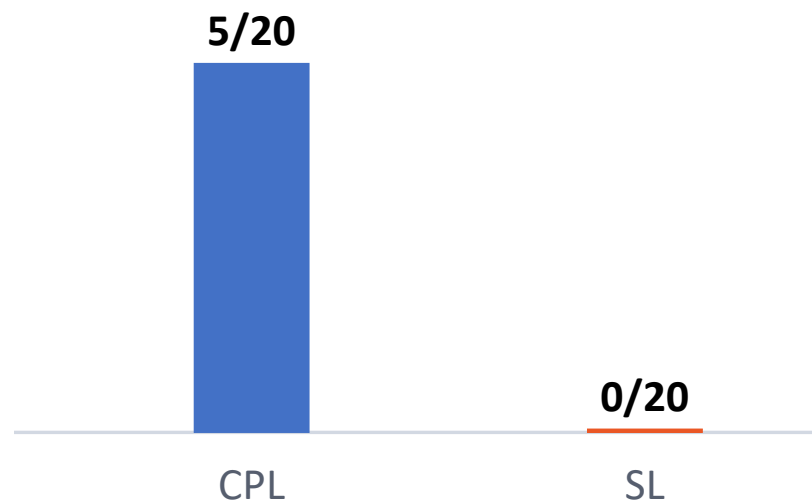
Average verified theorems per run

20 runs; same token budget



Target theorem discovered

Fisher's exact test $p = 0.024$

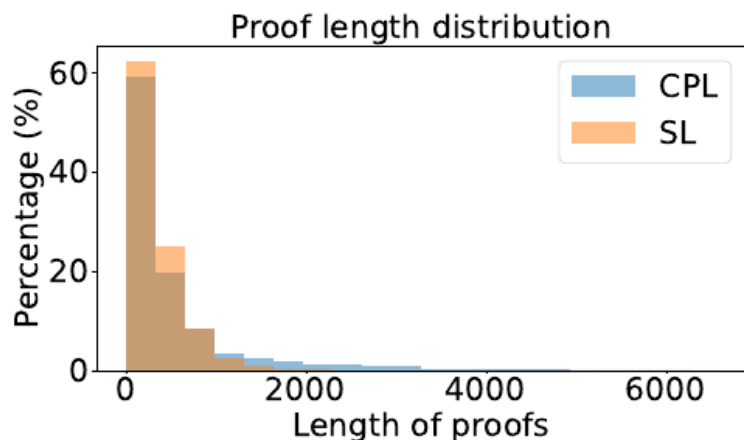


Interpretation: SL produces more verified theorems, but CPL discovers the hard target theorem.

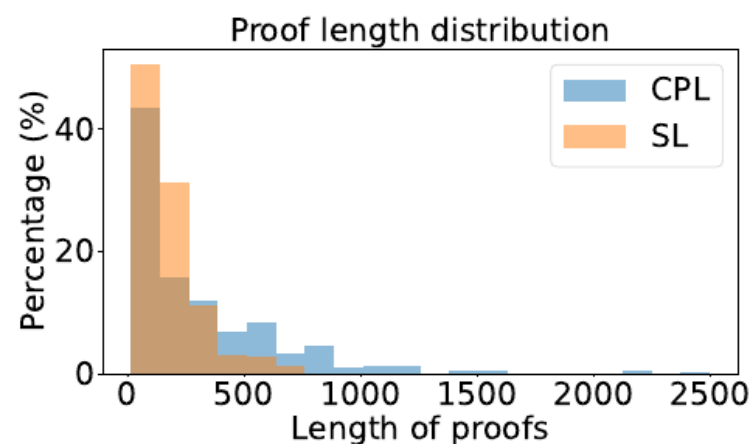
Proof length evidence

Longer Lean proofs support the claim that CPL reaches harder proof-search regimes.

With generated context



Without generated context

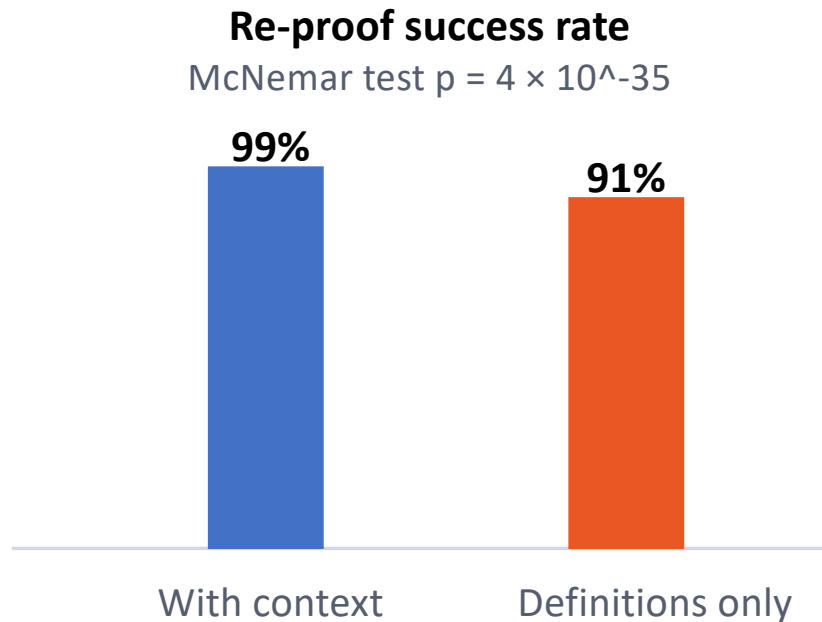


CPL shifts mass from only short proofs toward a longer proof tail.

The effect appears with and without generated context.

Generated context improves proof success

Re-proving 2,123 generated theorems isolates the value of the generated library as context.



**2106/2123
with context**

**1935/2123
definitions only**

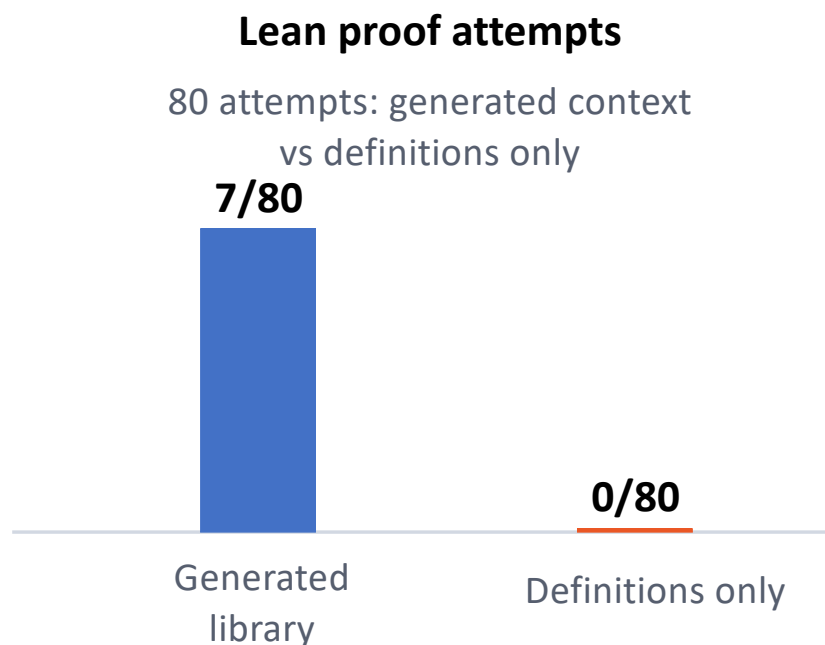
The context can help in two ways:

- available lemmas
- local proof strategies

Result: the LLM becomes a better prover in the local theory without fine-tuning.

Stress test: alpha-open intersection

The target theorem is difficult without the generated proof context.



**Natural-language proof attempts
from only the definitions**

GPT-4o
10 false; 6 incorrect
proofs

GPT-o3
16 false; 0 proofs

Generated context makes a proof strategy accessible, not merely a known theorem name.

What the generated proof tells us

Appendix A gives a verified Lean proof of the target theorem.

```
theorem P2_inter {X : Type*}
[TopologicalSpace X] {A B : Set X} :
  P2 A → P2 B → P2 (A ∩ B) := by
  intro hP2A hP2B x hx
  rcases hx with ⟨hxA, hxB⟩
  -- build neighbourhoods, use
  closure/interior reasoning
  ...
```

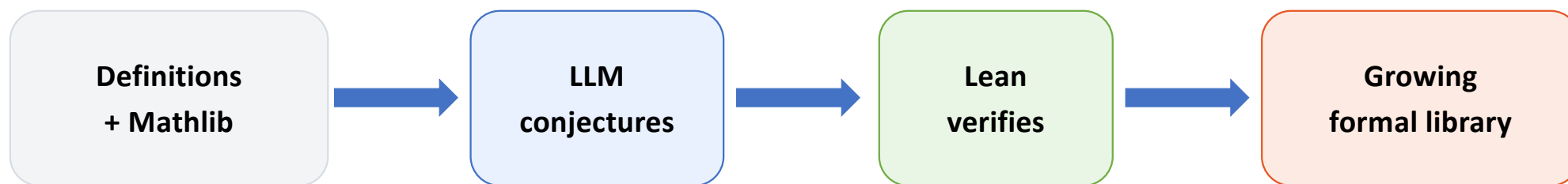
It differs from the original proof by Njastad.

It does not use other generated theorems as lemmas.

So the context likely teaches a proof style, not just a missing fact.

Implications and future work

The vision is automatic expansion of formal mathematics libraries under formal verification.



Implication

Closed-source LLMs can improve in practice through context, even when the model cannot be trained.

Future work

Guide conjecture generation toward deeper, more insightful, and less explored mathematical statements.

CPL provides a verified loop; the next challenge is better mathematical exploration.

Takeaways

A compact summary for a 20-minute presentation.

Separate

Conjecturing from proving

Reuse

Verified outputs as context

Verify

Speculation becomes library growth

Thank you.

SINICX